

HARP: released code vs. published paper

A point-by-point comparison of the experimental setup

August 5, 2026

Purpose. We are rerunning HARP’s released code on our own generated data, so that our comparison against their published numbers is like-for-like. While reading their code to build the data adapter, we found one place where the released implementation does not match the setup described in the paper. This document lists what we checked, with exact quotes and file/line references, so it can be verified independently.

Summary. The *method* is implemented exactly as the paper describes – detector architecture, token aggregation, optimiser, and correctness thresholds all match. The one substantive difference is in **how the data is split into training and test sets**.

The one difference that matters, in plain terms

What is *not* wrong. The overall structure is right, and this should be stated first to avoid overstating the finding. Questions are correctly labelled “known” or “unknown” according to the paper’s definition; all unknown questions are correctly placed in the test set and never used for training; and the 75/25 ratio is correctly applied only to the known group. There is a genuine known/unknown separation.

What is wrong. It is one level further down. Each question is answered ten times. The paper says the 75/25 split within the known group is made *over questions*. The code instead pools that group’s answers into a single list and splits *that* at random.

The practical effect: a known question typically has around seven or eight of its ten answers in training and the remaining two or three in the test set. Because the shuffle is random rather than a clean per-question allocation, a known question keeps all ten answers on the training side only about 5.6% of the time (0.75^{10}) and almost never lands entirely in the test set. So roughly **94% of known questions contribute answers to both sides**, and for those the question is not unseen at test time – the detector was trained on other answers to it.

The unknown questions are unaffected: none of their answers appear in training, so that portion of the test set is genuinely unseen.

This matters for our project specifically, because our own analysis shows that a large share of what these detectors appear to measure is *which questions the model finds hard*, rather than whether a particular answer is wrong. A split that puts the same question on both sides makes that shortcut available to the detector.

How large is the effect likely to be? It applies only to the “25% of known” part of the test set; the $\mathcal{X}_{unknown}$ questions are genuinely unseen. So the impact should scale with how much of the test set comes from answerable questions. Using our own generations as a guide: on Qwen/TruthfulQA roughly half the test answers would come from questions seen in training, whereas on Qwen/NQ-Open only about 3% would. If the effect is real, it should therefore inflate TruthfulQA and TyDiQA-GP noticeably and NQ-Open barely.

Our own implementation follows the paper, not the code. We split by question. This is visible in our logs: for LLaMA/TruthfulQA we get 3,870 training answers ($387 \text{ questions} \times 10$) and 4,300 test answers ($430 \text{ questions} \times 10$), summing to exactly 817 questions. Every count is a clean multiple of ten. So our current comparison holds our method to a stricter standard than their published numbers were produced under.

A separate note: the LLaMA model differs from ours

This is *not* a paper/code discrepancy – the two agree – but it affects our comparison. The paper (§5.1) evaluates “Qwen-2.5-7B-Instruct and LLaMA-3.1-8B,” and the code confirms it at `main.py:20`, where the LLaMA entry points to `meta-llama/Llama-3.1-8B` – the **base** model, not the instruction-tuned one. Our generations used the Instruct variant, **Meta-Llama-3.1-8B-Instruct**. The Qwen model matches exactly. Any like-for-like claim should therefore start with Qwen.

What we propose

1. Run their code **exactly as released** on our generated data. We supply the data through the cache files their pipeline already reads (`main.py:120--123`), so nothing in their method is touched. If our numbers land close to their published ones, that confirms both the handoff and the reading above.
2. Add a matching answer-level split option to **our** evaluator, so the two systems are compared under identical conditions. This changes only our code, never theirs.
3. Treat this document as provisional until step 1 is done. The reading of the code is straightforward, but a claim of this kind should rest on a reproduction rather than on a code inspection alone.

We would also note that this looks like a mismatch between the write-up and the released implementation rather than anything deliberate: the paper’s own formalism is precise, and the method itself is implemented faithfully.

Item	What the paper says	What the code does	Match?
Known / unknown grouping	§3.1: a question is “known” if any of its generated answers matches the reference. Appendix A: the unknown questions are all placed in the test set.	Correct. <code>get_known.py:115</code> marks a question known if any of its 10 answers is judged correct; <code>main.py:264</code> puts <i>every</i> unknown question into the test set and none into training; the 75/25 ratio is applied only to the known group.	Yes
How the known group is split	§3.1: “Let $\mathcal{X}_{known} = \{x \mid known(x) = 1\}$ denote the set of all inputs whose knowledge is known to the LLMs.” Appendix A: “75% of $\mathcal{X}_{known}$ is used for training, while the remaining 25%, together with $\mathcal{X}_{unknown}$, is used to test the hallucination detector on unseen data .”	Within the known group, splits individual answers , not questions. <code>main.py:184--201</code> builds one entry per answer (loop over questions, then over that question’s 10 answers); <code>get_hidden_state.py:72--79</code> stores one vector per answer; <code>main.py:237--246</code> zips these into one list; <code>main.py:259</code> splits that list 75/25 with shuffling, which in <code>utils.py</code> is a <code>torch.randperm</code> – a random shuffle of the answer list.	No
Answers per question	Appendix A: “beam search is used to generate 10 answer paths per question.”	10 requested (<code>main.py:157--158</code>), but answers that decode to empty text are dropped (<code>get_known.py:93--94</code>), so a question can end up with fewer than 10.	Minor
Detector architecture	Appendix A: “a two-layer MLP with hidden dimension 1024 and ReLU activation.”	<code>mlp_trainer.py:27--31</code> : <code>Linear(input,1024) → ReLU → Linear(1024,1) → Sigmoid</code> .	Yes
Score for a QA pair	Eq. 16: “The maximum score among all tokens is taken as the hallucination score of the QA pair.”	<code>mlp_trainer.py:56, 192, 213</code> : <code>masked_scores.max(dim=-1)</code> – maximum over the answer’s tokens, with padding masked out.	Yes
Training setup	Appendix A: “50 epochs with the Adam optimizer, initial learning rate 1e-4, cosine decay, batch size 128, and weight decay 3e-4.”	<code>mlp_trainer.py:282</code> : <code>Adam(lr=lr, weight_decay=weight_decay)</code> ; <code>mlp_trainer.py:12</code> : <code>CosineAnnealingLR</code> . Values passed from <code>main.py</code> (1e-4, 3e-4, 50 epochs); batch size 128 comes from the README command.	Yes
Correctness label	§5.1: “An answer is considered correct if its ROUGE-L score exceeds 0.7 or its semantic similarity exceeds 0.5.”	<code>DatasetJudge.py:106--112</code> : same two thresholds, plus one extra condition the paper does not mention – the best similarity to a correct answer must exceed the best similarity to a wrong answer by more than a margin. That margin defaults to -1 (<code>DatasetJudge.py:26</code>), which makes the condition almost always true, so in practice it rarely changes a label.	Yes (minor undocumented extra)
Which layer is used	Not stated. §4.4 says only that “deep hidden states exhibit stronger features in the reasoning subspace.”	<code>main.py:210</code> requests a single key, <code>model.layers.[L-1]</code> where L is the model’s layer count – i.e. the final layer only.	Paper does not specify